Jenkins CI/CD Pipeline Setup

04/25/2026 08:00:36

Ci cd pipeline Jenkins configure

Setting up a CI/CD pipeline using Jenkins is a very common interview and real-world...

Yes

Alright — now let's build a real-world, production-style CI/CD pipeline using: Jenkins...

What about nightly build configuration

Nightly builds are a very common real-world requirement, and interviewers love this topi...

Ci cd pipeline Jenkins configure

4/25/2026, 7:26:40 AM

Setting up a **CI/CD pipeline using Jenkins** is a very common interview and real-world requirement. I'll walk you through it step-by-step in a **practical, production-style flow** (not just theory).

🔷 1. High-Level CI/CD Flow

```
Developer → Git Push → Jenkins Trigger → Build → Test → Package → Deploy
```

Example:

- Code pushed to GitHub
- Jenkins detects change
- Runs build (Maven/Gradle)
- Executes tests
- Builds JAR/Docker image
- Deploys to server (Tomcat/Kubernetes)

🔷 2. Jenkins Setup (Basic)

Step 1: Install Jenkins

- Download from Jenkins official site
- Run on port `8080`

Step 2: Install Required Plugins

Go to:

```
Manage Jenkins → Plugins
```

Install:

- Git Plugin

- Maven Integration Plugin
- Pipeline Plugin
- Docker Plugin (if using Docker)
- Kubernetes Plugin (if needed)

🔷 3. Configure Global Tools

```
Manage Jenkins → Global Tool Configuration
```

Add:

- JDK
- Maven
- Git

🔷 4. Create a Pipeline Job

```
New Item → Pipeline → Enter Name
```

🔷 5. Pipeline Script (Jenkinsfile)

Here's a **real CI/CD pipeline example (Spring Boot app)**:

```
pipeline {
    agent any

    tools {
        maven 'Maven3'
        jdk 'JDK17'
    }

    environment {
        IMAGE_NAME = "myapp"
    }

    stages {
```

```
        stage('Checkout Code') {
            steps {
                git 'https://github.com/your-repo.git'
            }
        }

        stage('Build') {
            steps {
                sh 'mvn clean install'
            }
        }

        stage('Test') {
            steps {
                sh 'mvn test'
            }
        }

        stage('Package') {
            steps {
                sh 'mvn package'
            }
        }

        stage('Build Docker Image') {
            steps {
                sh 'docker build -t myapp:latest .'
            }
        }

        stage('Deploy') {
            steps {
                sh 'docker run -d -p 8081:8080 myapp:latest'
            }
        }
    }
}
```

🔷 6. GitHub Integration (Auto Trigger)

Option 1: Poll SCM

```
H/5 * * * *
```

Option 2: Webhook (Recommended)

- Go to GitHub repo
- Settings → Webhooks
- Add:

```
http://<jenkins-url>/github-webhook/
```

🔷 7. Credentials Setup

```
Manage Jenkins → Credentials
```

Add:

- GitHub token
- DockerHub credentials
- SSH keys (for server deployment)

🔷 8. Deployment Options

🔶 Simple

- Run JAR:

```
java -jar app.jar
```

🔶 Docker (Recommended)

```
docker build -t app .
docker run -d -p 8080:8080 app
```

🔶 Kubernetes (Advanced)

- Use Helm / kubectl
- Jenkins triggers deployment YAML

🔷 9. Real Interview Flow (IMPORTANT ⭐)

If interviewer asks:

👉 “Explain CI/CD pipeline using Jenkins”

Say:

> *“When developer pushes code to Git, Jenkins is triggered via webhook. Jenkins pulls the code, builds it using Maven, runs test cases, packages it into a JAR or Docker image, and deploys it to the target environment like a server or Kubernetes cluster.”*

🔷 10. Common Interview Questions

Q1: Why Jenkins?

- Automation of build, test, deploy
- Reduces manual effort
- Faster delivery

Q2: What is Jenkinsfile?

- Pipeline as Code (Groovy script)

Q3: Difference: Freestyle vs Pipeline?

- Freestyle → UI based
- Pipeline → Code based (recommended)

Q4: How do you handle failures?

- Post block:

```
post {
    failure {
        echo "Build Failed"
    }
}
```

🔷 11. Pro-Level Add-ons (for interviews 🚀)

You can mention:

- SonarQube → Code quality
- Nexus → Artifact storage
- Docker → Containerization
- Kubernetes → Deployment
- Slack/Email → Notifications

🔥 Real-World Architecture

```
GitHub → Jenkins → Maven Build → SonarQube → Docker → Kubernetes
```

💡 If you want next level

I can give you:

- ↪ Full **Spring Boot + Jenkins + Docker + Kafka CI/CD**
- ↪ Complete **system design diagram**
- ↪ Real **production YAML + configs**

Just say:

👉 “Give me end-to-end CI/CD with microservices”

Yes

4/25/2026, 7:28:02 AM

Alright — now let's build a **real-world, production-style CI/CD pipeline** using:

- Jenkins
- Docker
- Apache Kafka
- Kubernetes
- Spring Boot

This is exactly the kind of **architecture interviewers expect at Nagarro / product companies**.

🚀 1. High-Level Architecture (Microservices + CI/CD)

```
Client → API Gateway → Order Service → Kafka → Payment Service
                            ↓
                        Database

GitHub → Jenkins → Build → Test → Docker → Push → Kubernetes Deploy
```

🔷 2. Microservices Flow (with Kafka)

Example:

- User places order
- Order Service publishes event to Kafka
- Payment Service consumes event

🔶 Kafka Producer (Order Service)

```
@RestController
@RequestMapping("/orders")
public class OrderController {
```

```
    @Autowired
    private KafkaTemplate<String, String> kafkaTemplate;

    @PostMapping
    public String createOrder(@RequestBody String order) {
        kafkaTemplate.send("order-topic", order);
        return "Order Placed";
    }
}
```

🔶 Kafka Consumer (Payment Service)

```
@Component
public class PaymentConsumer {

    @KafkaListener(topics = "order-topic", groupId = "group1")
    public void processPayment(String order) {
        System.out.println("Processing payment for: " + order);
    }
}
```

🔷 3. Dockerize Spring Boot App

Dockerfile

```
FROM openjdk:17
COPY target/app.jar app.jar
ENTRYPOINT ["java","-jar","/app.jar"]
```

🔷 4. Kubernetes Deployment YAML

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: order-service
spec:
```

```
  replicas: 2
  selector:
    matchLabels:
      app: order-service
  template:
    metadata:
      labels:
        app: order-service
    spec:
      containers:
      - name: order-service
        image: myrepo/order-service:latest
        ports:
        - containerPort: 8080
```

🔷 5. Jenkins Pipeline (End-to-End)

This is the **main thing interviewers want** 👇

```
pipeline {
    agent any

    environment {
        IMAGE_NAME = "myrepo/order-service"
        TAG = "latest"
    }

    tools {
        maven 'Maven3'
        jdk 'JDK17'
    }

    stages {

        stage('Checkout') {
            steps {
                git 'https://github.com/your-repo.git'
            }
        }

        stage('Build') {
            steps {
                sh 'mvn clean install'
            }
```

```
        }

        stage('Test') {
            steps {
                sh 'mvn test'
            }
        }

        stage('Code Quality') {
            steps {
                sh 'mvn sonar:sonar'
            }
        }

        stage('Build Docker Image') {
            steps {
                sh 'docker build -t $IMAGE_NAME:$TAG .'
            }
        }

        stage('Push to Docker Hub') {
            steps {
                withCredentials([string(credentialsId: 'docker-pass', variable:
'PASS')]) {
                    sh 'docker login -u user -p $PASS'
                    sh 'docker push $IMAGE_NAME:$TAG'
                }
            }
        }

        stage('Deploy to Kubernetes') {
            steps {
                sh 'kubectl apply -f deployment.yaml'
            }
        }
    }

    post {
        success {
            echo 'Deployment Successful'
        }
        failure {
            echo 'Pipeline Failed'
        }
```

```
    }
}
```

🔷 6. Complete Flow (Explain in Interview ⭐)

Say this confidently:

> *"Developer pushes code to GitHub → Jenkins triggers via webhook → Jenkins builds using Maven → runs tests → performs SonarQube analysis → builds Docker image → pushes to Docker Hub → deploys to Kubernetes → application becomes live."*

🔷 7. Important Configurations

🔶 application.yml (Kafka)

```yaml
spring:
  kafka:
    bootstrap-servers: localhost:9092
    consumer:
      group-id: group1
```

🔶 Jenkins Credentials

- DockerHub username/password
- GitHub token
- Kubernetes config (kubeconfig)

🔥 8. Advanced Enhancements (VERY IMPORTANT)

Mention these to stand out:

✅ Blue-Green Deployment

- Zero downtime deployment

✅ Rolling Updates (Kubernetes default)

- Gradual replacement of pods

✅ Monitoring

- Prometheus + Grafana

✅ Logging

- ELK Stack

🔥 9. Real System Design Question (Interview Level)

👉 “Design CI/CD pipeline for e-commerce system”

You should say:

- Microservices (Order, Payment, User)
- Async communication using Kafka
- CI/CD using Jenkins
- Containerization using Docker
- Deployment using Kubernetes
- Monitoring + logging added

💡 If you want next level (HIGHLY RECOMMENDED)

I can give you:

- 🔥 ↳ Full **Spring Boot project structure (ready to run)**
- 🔥 ↳ **Docker + Kubernetes + Jenkins config files**
- 🔥 ↳ **Interview mock Q&A (very tricky scenarios)**
- 🔥 ↳ **Failure scenarios (what happens if Kafka/Jenkins fails)**

4/25/2026, 7:28:03 AM

Just say:

👉 “Give me failure scenarios + debugging CI/CD”

What about nightly build configuration

4/25/2026, 7:28:41 AM

Nightly builds are a **very common real-world requirement**, and interviewers love this topic because it shows you understand automation beyond simple CI triggers.

Let's break it down clearly.

🌙 What is a Nightly Build?

A **nightly build** is a job that runs automatically at a specific time (usually midnight) to:

- Pull latest code
- Build the application
- Run full test suite (including heavy tests)
- Generate reports
- Sometimes deploy to QA/UAT

👉 Unlike webhook builds (triggered on every commit), this is **time-based scheduling**.

🔷 Why Nightly Builds?

In real projects:

- Running full tests on every commit is slow ❌
- Some tests are expensive (integration, performance) ❌

So we do:

Build Type	When	Purpose
CI Build	On every commit	Quick validation
Nightly Build	Scheduled	Full regression testing

🔷 How to Configure in Jenkins

✅ Option 1: Using UI (Freestyle or Pipeline)

Go to:

```
Job → Configure → Build Triggers
```

Enable:

```
✔ Build periodically
```

🕒 Cron Syntax (VERY IMPORTANT ⭐)

Example:

```
H 0 * * *
```

👉 Meaning:

- Runs once daily at midnight

More Examples

Cron	Meaning
`H 2 * * *`	Every day at 2 AM
`H H * * *`	Random time daily (load balancing)
`H */6 * * *`	Every 6 hours

🔷 Option 2: Pipeline (Jenkinsfile)

This is what interviewers expect 👇

```
pipeline {
    agent any

    triggers {
        cron('H 0 * * *')
    }

    stages {
```

```
        stage('Checkout') {
            steps {
                git 'https://github.com/your-repo.git'
            }
        }

        stage('Build') {
            steps {
                sh 'mvn clean install'
            }
        }

        stage('Full Regression Tests') {
            steps {
                sh 'mvn verify'
            }
        }

        stage('Generate Report') {
            steps {
                sh 'mvn site'
            }
        }
    }
}
```

🔷 Real Nightly Build Flow

```
12:00 AM → Jenkins triggers job
↓
Pull latest code
↓
Build + Full test suite
↓
Generate reports
↓
Send email/Slack notification
```

🔥 Real-Time Use Cases (Interview GOLD ⭐)

1. Regression Testing

- Run all test cases overnight

2. Data Sync Jobs

- Batch jobs (Spring Batch)

3. Report Generation

- Business reports generated daily

4. Database Cleanup

- Archive old records

🔷 Nightly Build + Spring Batch (VERY IMPORTANT)

You can combine with **Spring Batch jobs**:

```
Jenkins → Trigger → Spring Batch Job → Process large data
```

Example:

- Process 1 million transactions at night
- Generate reports
- Store in DB

🔷 Notifications (Production Setup)

Add in pipeline:

```
post {
    success {
        echo 'Nightly Build Passed'
    }
    failure {
        echo 'Nightly Build Failed'
        mail to: 'team@company.com',
             subject: 'Nightly Build Failed',
             body: 'Check Jenkins logs'
```

```
    }
}
```

🔷 Best Practices (IMPORTANT)

✅ Use separate job for nightly build

- Don’t mix with CI job

✅ Run heavy tests only here

- Integration / performance tests

✅ Use `H` in cron

- Prevents server overload

✅ Store artifacts

- For debugging failures

🔥 Interview Answer (Perfect)

If asked:

👉 “How do you configure nightly builds in Jenkins?”

Say:

> *“We use cron-based triggers in Jenkins pipeline. For example, `H 0 * * *` schedules the job at midnight. The nightly job runs full regression tests, builds the application, and generates reports. It is separate from CI jobs to avoid slowing down developer feedback.”*

💡 Want to go deeper?

I can give you:

- 🔥 ↪ Nightly build with **Spring Batch + Kafka**
- 🔥 ↪ Real **failure scenarios (job failed at 2 AM, what will you do?)**
- 🔥 ↪ **Production debugging checklist**

Just say:

👉 “Give me failure + debugging scenarios”